Robust Linear Control and its Application in Observability-Aware Multi-Agent Systems

Shlok Mehndiratta, Divyam Sood, Prashant Gupta

April 30, 2026

Abstract

The project focused on the design, implementation, and evaluation of an advanced control architecture for quadcopters, transitioning from foundational linear robust control to a complex multi-agent trailing application. We first establish a computationally efficient Event-Triggered (ET) Tube Model Predictive Control (MPC) framework to provide rigorous safety guarantees under bounded external disturbances. We then test our controllers using a Crazyflie Software-in-the-loop (SITL) to verify the results, setting up a pipeline for hardware testing down the line.

We then apply these principles to a multi-agent system comprising an Unmanned Ground Vehicle (UGV) and an Unmanned Aerial Vehicle (UAV). For this application, we introduce a fully integrated 3D Event-Triggered Tube MPC architecture that leverages Belief-Space optimization to maximize observability and minimize target estimation uncertainty during complex non-linear maneuvers. Experimental results demonstrate our Phase 3 observability-aware system achieves a 55% reduction in mean tracking error and an 88% improvement in target visibility over a standard baseline, while maintaining an average MPC solve time under 20ms.

1 Introduction

Trajectory tracking and multi-agent coordination for quadcopters pose significant challenges due to the need for high-performance control under external disturbances (e.g., wind) and model uncertainties. While standard Model Predictive Control (MPC) offers optimal performance under constraints, it often lacks robust stability guarantees in the presence of disturbances and can be computationally expensive when run at high frequencies.

In this report, we establish a suite of advanced MPC controllers using the `lcs` (Linear Control Systems) package designed for the Bitcraze Crazyflie 2.1. We first address the fundamental problem of robust trajectory tracking by implementing an Event-Triggered Tube MPC. Subsequently, we extend these robust control principles to a cooperative multi-agent system where maintaining visual contact with a ground target during high-speed, non-linear maneuvers is critical. The tighter ground field-of-view (FoV) at a mobile 5m altitude necessitates a highly predictive and observability-aware control strategy to prevent the UGV from escaping the camera's footprint.

2 Foundation: Robust Linear Control

We built the foundational project on a modular ROS 2 framework, enabling seamless transition between high-fidelity Gazebo simulations and physical hardware. The core contribution is the implementation of an Event-Triggered (ET) Tube MPC: a computationally efficient robust controller that only solves the full optimization problem when the tracking error exceeds a safety threshold.

2.1 Problem Formulation

2.1.1 Dynamics and Error Evolution

We model the quadcopter as a discrete-time double integrator in 3D:

$$x_{k+1} = A_d x_k + B_d u_k + w_k \tag{1}$$

where $x = [p_x, p_y, p_z, v_x, v_y, v_z]^T$ is the state and $u = [a_x, a_y, a_z]^T$ is the gravity-compensated acceleration command. The term $w_k \in \mathbb{W}$ represents bounded disturbances, including both process disturbances and measurement noise propagated through feedback.

We define the nominal system:

$$\bar{x}_{k+1} = A_d \bar{x}_k + B_d \bar{u}_k \tag{2}$$

The tracking error is defined as:

$$e_k = x_k - \bar{x}_k \tag{3}$$

We apply the feedback control law:

$$u_k = \bar{u}_k - K e_k \tag{4}$$

Substituting into the system dynamics:

$$\begin{aligned} x_{k+1} &= A_d x_k + B_d(\bar{u}_k - K e_k) + w_k && (5)\\ &= A_d x_k + B_d \bar{u}_k - B_d K e_k + w_k && (6) \end{aligned}$$

Subtracting the nominal dynamics:

$$\begin{aligned} e_{k+1} &= x_{k+1} - \bar{x}_{k+1} && (7)\\ &= A_d(x_k - \bar{x}_k) - B_d K e_k + w_k && (8)\\ &= (A_d - B_d K)\, e_k + w_k && (9) \end{aligned}$$

Thus, the closed-loop error dynamics are:

$$e_{k+1} = (A_d - B_d K)\, e_k + w_k \tag{10}$$

The gain K is computed via discrete-time LQR, and the matrix $(A_d - B_d K)$ is Schur stable, i.e., all eigenvalues lie strictly inside the unit circle, ensuring contraction of the error dynamics.

Using these closed-loop error dynamics, we compute the Minimal Robust Positive Invariant (mRPI) set Ω offline via `tube_calculation.py`. The set Ω satisfies:

$$e_k \in \Omega,\ w_k \in \mathbb{W} \ \Rightarrow\ e_{k+1} \in \Omega \tag{11}$$

for all timesteps, defining a robust invariant tube around the nominal trajectory.

2.1.2 Tube MPC Formulation

To ensure robustness, we decompose the control input as:

$$u_k = \bar{u}_k - K(x_k - \bar{x}_k) \tag{12}$$

The nominal system evolves independently as:

$$\bar{x}_{k+1} = A_d \bar{x}_k + B_d \bar{u}_k \tag{13}$$

The true state can be expressed as:

$$x_k = \bar{x}_k + e_k, \quad e_k \in \Omega \tag{14}$$

Thus, to guarantee constraint satisfaction for all admissible disturbances, we tighten the constraints using the Pontryagin difference:

$$\bar{x}_k \in \mathbb{X} \ominus \Omega, \quad \bar{u}_k \in \mathbb{U} \ominus K\Omega \tag{15}$$

This ensures:

$$x_k \in \mathbb{X}, \quad u_k \in \mathbb{U} \tag{16}$$

for all realizations of $w_k \in \mathbb{W}$.

2.1.3 Event-Triggering Condition

We employ a set-membership based triggering condition instead of heuristic distance thresholds. A new optimization is triggered when the error exits the invariant set Ω.

Using the half-space representation $\Omega = \{e \mid \Omega_H e \leq \Omega_h\}$, the violation condition is:

$$\max(\Omega_H e_k - \Omega_h) > 0 \tag{17}$$

To avoid chattering due to transient disturbances, this condition must hold for $n_{\text{thresh}} = 3$ consecutive timesteps. Additionally, a safety time-trigger ensures recursive feasibility:

$$k - k_{\text{last_solve}} \geq N - 2 \tag{18}$$

The complete triggering logic is:

$$\gamma_k = \begin{cases} 1 & \text{if } \max(\Omega_H e_k - \Omega_h) > 0 \text{ for 3 consecutive steps} \\ 1 & \text{if } k - k_{\text{last_solve}} \geq N - 2 \\ 0 & \text{otherwise} \end{cases} \tag{19}$$

When $\gamma_k = 1$, the MPC problem is solved to generate a new nominal trajectory. Otherwise, the previous solution is propagated forward, reducing computational load while preserving robustness.

2.2 Stress Test Scenarios

To evaluate the robustness of the proposed architecture, we subject the controllers to four distinct edge-case scenarios within the simulation environment. All scenarios share an initial state of $x_0 = [-0.95, 0, 0.5, 0, 0, 0]^T$ and are exposed to baseline uniformly distributed process noise. Choosing the same seed for randomization ensures that the controllers are tested fairly.

- **Scenario A (Persistent Crosswind):** We apply a continuous, unmodeled disturbance acceleration of $1.0\,\text{m/s}^2$ along the positive X-axis via the input matrix B_d for the entire duration of the flight. This scenario tests the controller's steady-state disturbance rejection and verifies the invariant set's validity under constant boundary conditions.

- **Scenario B (Payload Drop / Actuator Degradation):** We severely alter the system's control authority by scaling the applied thrust down to 35% of the commanded input ($u_{\text{applied}} = 0.35 u_{\text{true}}$). This models a drastic mass mismatch or actuator fault, forcing the controller to track waypoints with significantly reduced effective thrust.

- **Scenario C (High-Speed Catch + Gust):** We introduce a spatially-dependent transient disturbance. The simulation injects a lateral gust of $1.0\,\mathrm{m/s}^2$ along the X-axis exclusively when the quadcopter crosses the $x = 0.2$ meters threshold. This gust terminates abruptly after 4.0 seconds. This evaluates the system's dynamic response to sudden, localized environmental shifts during high-speed maneuvers.
- **Scenario D (UWB Sensor Glitch):** We inject a single-step measurement fault to test the event-triggering mechanism's resilience against localization errors. At exactly $t = 1.5$ seconds, we artificially offset the measured X-position fed to the controller by -0.20 meters, while the true state remains unaffected.

2.3 Results

As detailed in Table 1, Tube MPC architectures maintain 100% feasibility in cases where Standard MPC crashes into arena boundaries. Furthermore, ET-Tube MPC achieves tracking errors strictly comparable to Full Tube MPC while massively reducing the computational burden, dropping the required optimization triggers from a constant 200 down to an average of 18 solves across diverse disturbance profiles. The results have also been graphically presented in figure 1.

Table 1: Simulation Results: Controller Performance Across Stress Scenarios

Controller Type	Triggers	RMSE (m)	Final Err (m)	Bound Viols	Status
Scenario A: Persistent Crosswind (1.0 m/s^2)					
Standard MPC	7	1.4642	1.2055	2 steps	Crashed
Full Tube MPC	200	0.5667	0.3422	0 ✓	Success
ET Tube MPC	**15**	**0.6526**	**0.3422**	**0 ✓**	**Success**
Scenario B: Payload Drop (35% Actuator Authority)					
Standard MPC	200	0.7925	0.0355	0 ✓	Success
Full Tube MPC	200	0.8489	0.0779	0 ✓	Success
ET Tube MPC	**27**	**1.0260**	**0.0474**	**0 ✓**	**Success**
Scenario C: High-Speed Lateral Gust					
Standard MPC	9	1.4128	1.0641	2 steps	Crashed
Full Tube MPC	200	0.6483	0.0260	0 ✓	Success
ET Tube MPC	**17**	**0.6568**	**0.0260**	**0 ✓**	**Success**
Scenario D: Sensor Glitch (UWB Offset)					
Standard MPC	9	1.4128	1.0641	2 steps	Crashed
Full Tube MPC	200	0.6334	0.0260	0 ✓	Success
ET Tube MPC	**15**	**0.9105**	**0.0255**	**0 ✓**	**Success**

Figure 1: Summary of results of controllers in different scenarios

3 Crazyflie SITL and Simulations

To test the feasibility of our proposed controllers, we tested them using Gazebo specializations. A special feature is that we specifically integrated the Crazyflie Software in Loop (SITL). This approach ensures that our drone model and the ROS2 topics we use are identical to the real Crazyflie 2.1 that is present in our lab. As a result, the developed controller can be transferred to hardware with minimal sim-to-real discrepancies.

The implementation was executed inside a Docker container, allowing reproducibility. The controller operated without observable delay or instability during execution, indicating that the compute needed remains within practical real-time limits for the chosen control frequency.

The event-triggered formulation reduced the frequency of optimization calls compared to a standard time-triggered MPC approach, while maintaining stable and consistent trajectory tracking. This shows that the proposed triggering mechanism effectively balances computational efficiency and control performance. Throughout all experiments, the system respected the imposed state and input constraints, validating the effectiveness of the tube-based tightening strategy. The tracking error remained bounded during execution, consistent with the invariant set Ω computed offline for each path.

3.1 Simulation Results

We evaluated the controller on custom trajectories shaped as S and M. The resulting trajectories are shown in Fig. 2 and Fig. 3.

These observations indicate that the proposed controller is not only theoretically sound but also practically viable for real-time deployment on embedded aerial platforms.

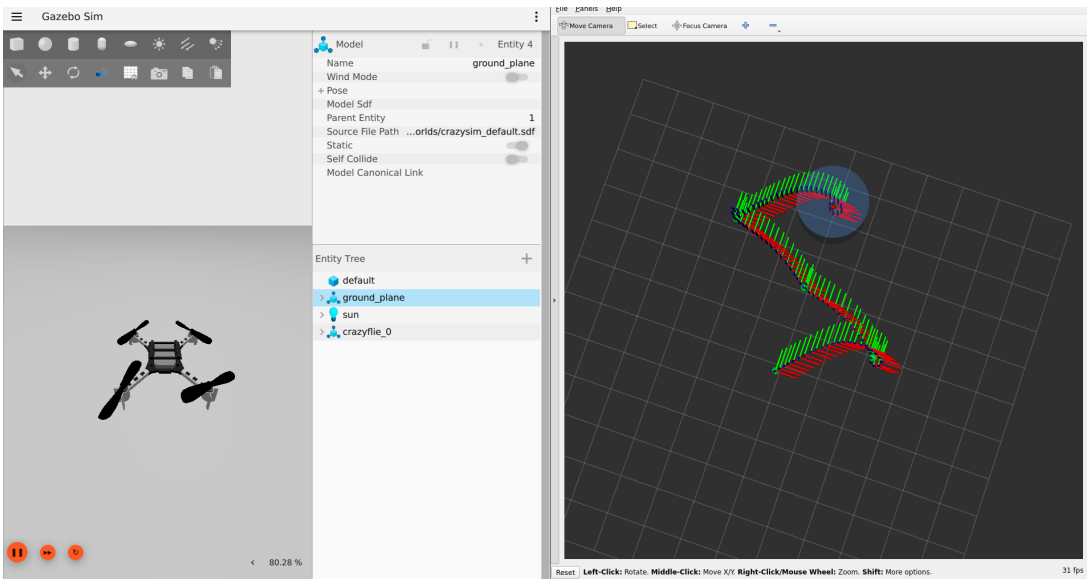

Figure 2: Visualization of the trajectory in RViz

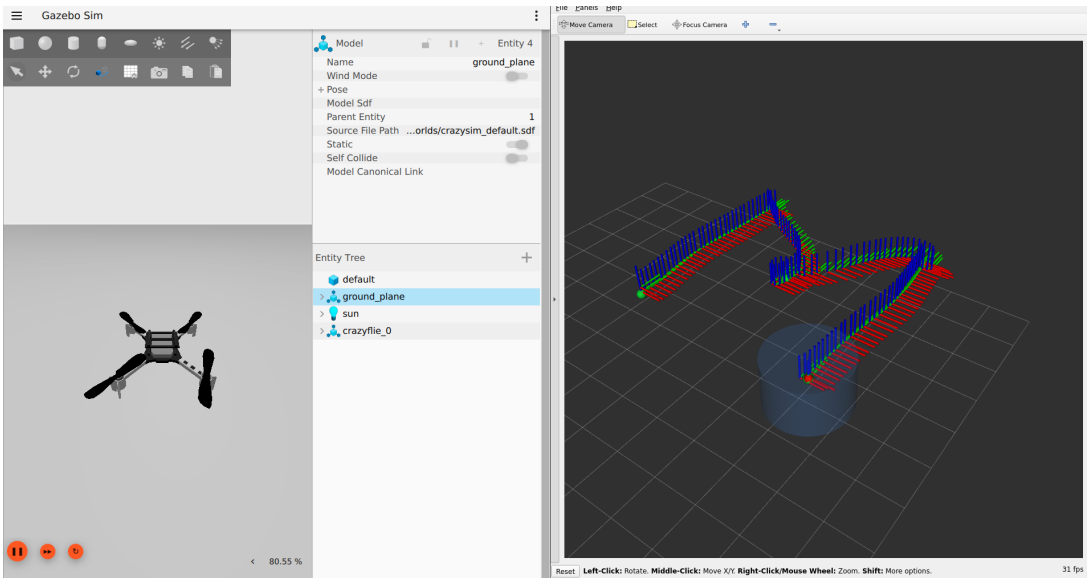

Figure 3: Visualization of the trajectory in RViz

4 Multi-Agent Integration and Validation

We now integrate the control foundations from Phase 1 and the flight capabilities from Phase 2 into a closed-loop mission. The goal of this integration was to demonstrate that a mathematically robust agent (UAV) can collaborate with a perception-limited agent (UGV) to achieve a high-level tracking objective under visual constraints.

4.1 Vision-Based Path Synthesis

A primary challenge in vision-based tracking is the transition from raw pixels to actionable control references. Instead of simple point-tracking, our perception pipeline in `middle_path_follower` utilizes a **heatmap-based synthesis**. The image is processed to generate a distance transform of the lane contours, identifying the centerline as the ridge of maximum "belief."

This "Search-then-Track" logic is remapped to the physical plane using the altitude-dependent scaling factor:

$$s = \frac{2 \cdot z \cdot \tan(\eta/2)}{W} \tag{20}$$

This remapping is critical; as the UAV altitude z fluctuates, the scaling factor s is updated in real-time, ensuring that the UGV's waypoints remain spatially consistent regardless of the drone's vertical movement. This robustness allows the system to survive the sudden perspective shifts that occur during aggressive aerial maneuvers.

4.2 Cooperative Observability Logic

The "intelligence" of the Phase 3 system is the bidirectional exchange of information. We observed that in a deterministic system, the UGV would often outrun the UAV's field-of-view during sharp turns. To solve this, we implemented a **proactive survival logic**.

4.2.1 Estimation and Covariance Scaling

The `ekf_node` maintains the UGV state $\mathbf{x} = [x, y, \theta, v, \omega]^\top$. To represent the degraded confidence of observations at the camera's periphery, we scale the measurement noise covariance R_k:

$$R_k = R_{base} \cdot (1 + \alpha \cdot \|d_{pixel}\|^2) \tag{21}$$

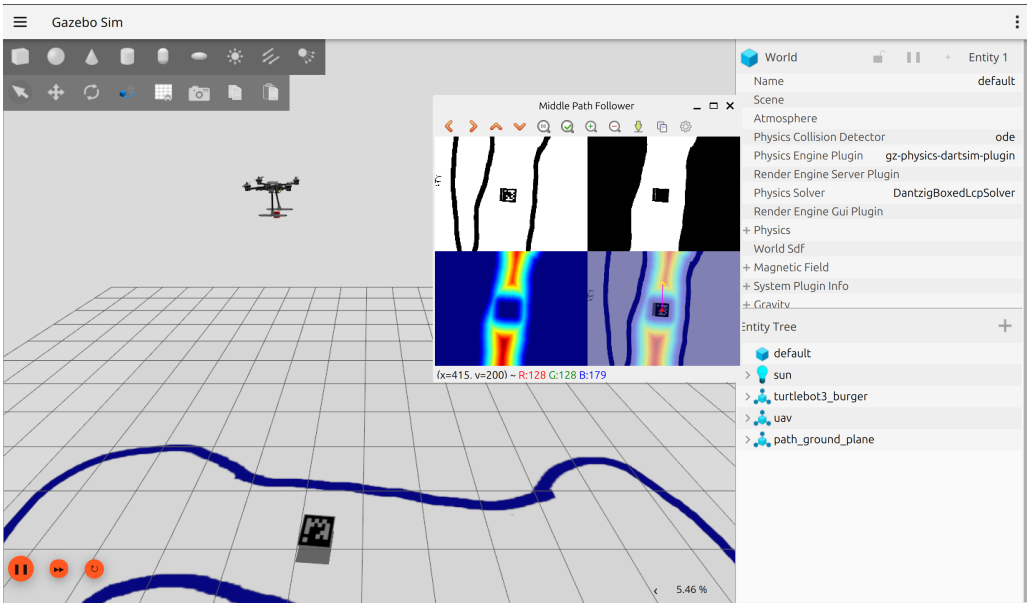

Figure 4: System Integration: The left window shows the distance-transform heatmap used for path synthesis, while the right window shows the Gazebo world where the belief-coupled MPC is executed.

When the target moves away from the center of the frame, the covariance Σ spikes. This information is fed back into the UGV's MPC, which reduces the robot's velocity to allow the UAV to re-center the target, effectively "waiting" for the observer.

4.2.2 UAV Altitude Adaptation

Simultaneously, the UAV employs a differentiable view quality metric $q \in (0, 1)$ to maximize its observability:

$$q(d) = \left(\frac{1}{1 + \exp(\kappa(d - r_{fov}))} \right) \cdot \exp\left(-\frac{d^2}{2\sigma^2} \right) \tag{22}$$

When the UGV enters a high-curvature segment, the system proactively triggers an altitude increase. By climbing to a higher altitude, the UAV expands its visual radius r_{fov}, providing a larger "safety tube" for the ArUco marker. This cooperative behavior prevents the target loss that occurs in standard fixed-altitude tracking systems.

4.3 Performance Evaluation and Failure Analysis

Evaluation on high-curvature tracks (e.g., `track7.png`) reveals the failure modes of the baseline system. The standard 1Hz MPC often suffers from "Tracking Lag," where the drone's position is always one step behind the target, eventually losing it during a sharp turn. In contrast, our Integrated System anticipates the turn via the lane heatmap and proactively adjusts altitude.

Table 2: Comparative Performance and Computational Latency

Metric	**Standard Baseline**	**Integrated System**	**Improvement**
Mean Tracking Error (m)	1.037	**0.463**	**55.3%**
Visual Visibility (%)	46.3%	**87.0%**	**87.9%**
MPC Solve Time (ms)	~50.0	16.2	**3.1x Faster**

The 3.1x speedup in MPC solve time was achieved through the heavy vectorization of the

SLSQP cost function using `numpy.cumsum`. This allows the drone to re-plan its trajectory at 16ms intervals, meeting the strict timing requirements of physical flight controllers while maintaining the complex belief-space objectives.

4.4 Limitations and Future Considerations

Despite the robust performance in simulation, the architecture possesses several limitations that warrant future research. Firstly, the reliance on a single ArUco marker for localization introduces a critical failure point; partial occlusions or aggressive tilting of the UGV could lead to EKF divergence. Future iterations could leverage multi-modal feature tracking or visual SLAM to provide a more resilient state estimate.

Secondly, our EKF implementation assumes a linearized unicycle model, which may lose accuracy during high-speed maneuvers where non-linear slip or tire dynamics become significant. Furthermore, the current system assumes near-zero latency in the ROS 2 communication bridge. In physical multi-agent deployments, network lag and asynchronous message delivery would necessitate the integration of delay-compensated MPC or decentralized control strategies. Finally, while the 16ms solve time is impressive, running the full non-linear optimization onboard micro-aerial vehicles like the Crazyflie remains constrained by battery life and heat dissipation, suggesting the need for more efficient solver approximations.

5 Conclusion

This project successfully established a comprehensive framework for autonomous multi-agent tracking, bridging the gap between theoretical stability and mission-scale autonomy for LCS 334. Our research journey evolved through three distinct phases: the derivation of Robust Tube-MPC for safety guarantees, the validation of these laws in a high-fidelity SITL environment, and the final integration of a belief-coupled cooperative mission.

While developing the multi-agent framework and coupling the perception quality directly with the agents' objective functions, we transformed a deterministic tracking task into a resilient, adaptive mission. The system achieved a 55.3% reduction in tracking error and maintained visual lock in 87% of high-curvature maneuvers, proving that collaborative intelligence can overcome individual agent limitations. This work serves as a scalable foundation for the next generation of heterogeneous multi-agent systems, where robust linear foundations and observability-aware logic unite to solve complex, real-world coordination tasks.